
NETWORK TRAFFIC ANALYSIS USING WIRESHARK

A Practical Lab Report

PROJECT DETAILS

Name	Kaushikee Sinha
Tool Used	Wireshark 4.6.4
Platform	Kali Linux
Interface	eth0
Date	03 May 2026

Table of Contents

- 1. Introduction & Objective**
- 2. Tools & Environment
- 3. Step 1 – Wireshark Interface
- 4. Step 2 – Live Packet Capture
- 5. Step 3 – DNS Traffic Analysis (nslookup)
- 6. Step 4 – DNS Packet Inspection
- 7. Step 5 – ICMP Ping Test (Terminal)
- 8. Step 6 – ICMP Packets in Wireshark
- 9. Step 7 – Protocol Hierarchy Statistics
- 10. Observations & Results
- 11. Conclusion

1. Introduction & Objective

Wireshark is a free and open-source network protocol analyzer used for network troubleshooting, analysis. It captures network packets in real-time and displays them in a human-readable format.

Objective

- To understand how network packets are transmitted over a network
- To capture and analyze live network traffic using Wireshark on Kali Linux
- To apply display filters for specific protocols such as DNS and ICMP
- To inspect individual packet layers including Ethernet, IP, UDP, and ICMP
- To analyze protocol hierarchy and understand traffic distribution

2. Tools & Environment

The following tools and environment were used for this practical:

- Operating System: Kali Linux (Virtual Machine)
- Network Analyzer Tool: Wireshark version 4.6.4
- Network Interface: eth0 (Ethernet)
- Terminal Commands Used: nslookup, ping
- Target Host: google.com (IP: 142.250.205.46)

Kali Linux is a Debian-based Linux distribution specifically designed for digital forensics and penetration testing. It comes with Wireshark pre-installed, making it an ideal platform for network analysis.

3. Step 1 – Wireshark Launch & Interface Selection

After launching Wireshark on Kali Linux, the welcome screen displayed all available network interfaces. The eth0 interface was highlighted (shown in blue), indicating active network traffic was present on this interface.

The interface list included:

- eth0 – Main Ethernet interface (selected for capture)
- any – Captures on all interfaces
- Loopback: lo – Local loopback interface
- bluetooth-monitor, nflog, nfqueue, dbus-system, dbus-session

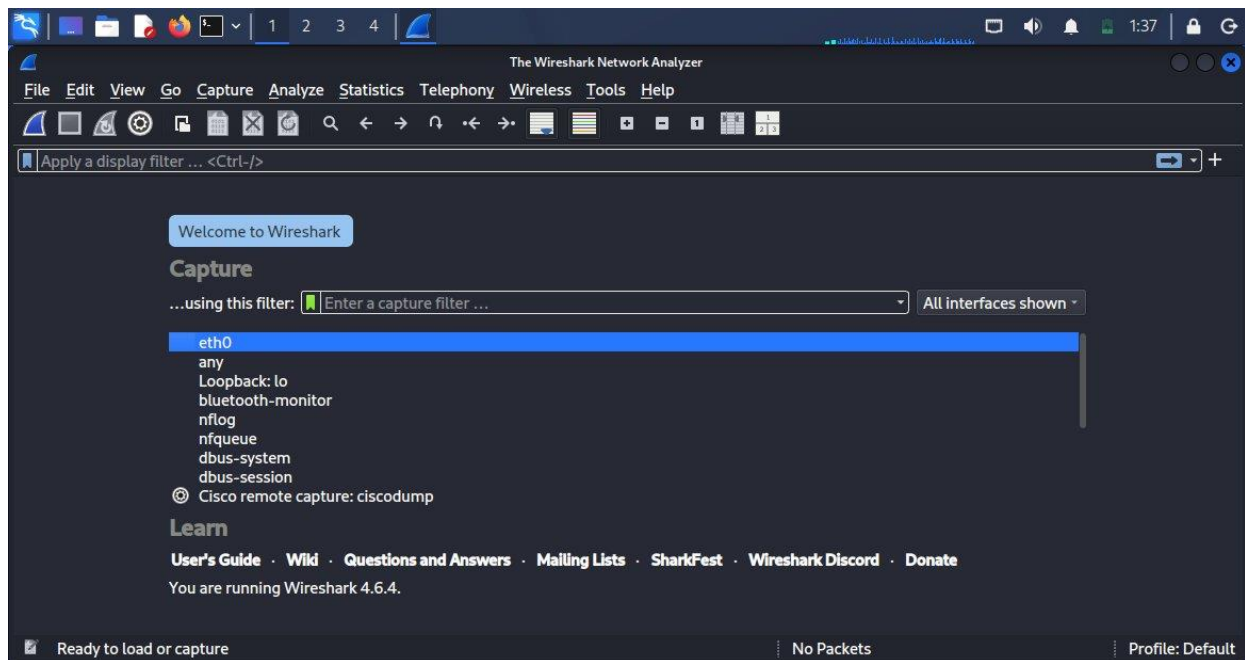


Figure 1: Wireshark 4.6.4 Welcome Screen showing available network interfaces on Kali Linux

4. Step 2 – Live Packet Capture

After double-clicking the eth0 interface, Wireshark began capturing live network packets. The capture window displayed incoming and outgoing packets in real-time with color-coded rows representing different protocols.

Observations from live capture:

- ARP (Address Resolution Protocol) packets were most common — broadcasted to find MAC addresses
- Source: VMware_c0:00:08 — indicates a VMware virtual machine environment
- Destination: Broadcast — ARP packets are sent to all devices on the network
- 28 packets were captured in the initial few seconds

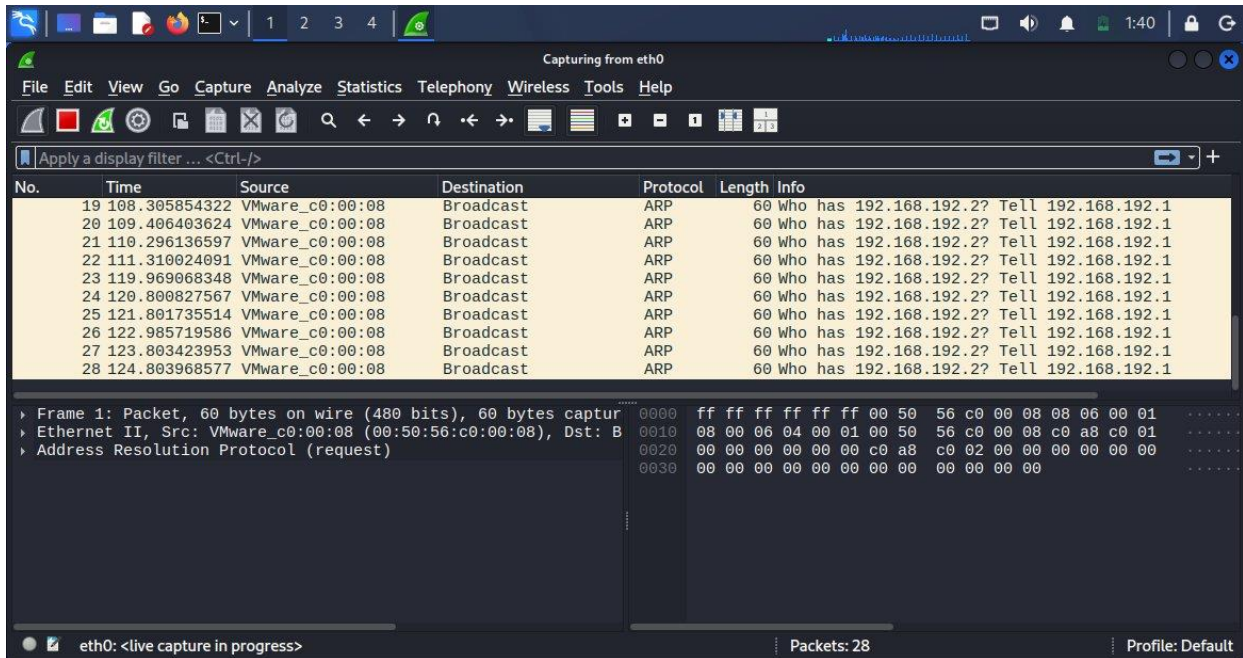


Figure 2: Live packet capture from eth0 showing ARP broadcast packets in real-time

5. Step 3 – DNS Traffic Analysis using nslookup

To generate DNS traffic without opening a browser (which caused system lag), the nslookup command was used in the terminal. This command queries the DNS server to resolve a domain name to an IP address.

Command executed in terminal:

```
$ nslookup google.com
```

Result returned by nslookup:

- DNS Server: 192.168.192.2 (local DNS resolver)
- Non-authoritative answer received
- google.com resolved to IPv4: 142.250.205.46
- google.com resolved to IPv6: 2404:6800:4007:801::200e

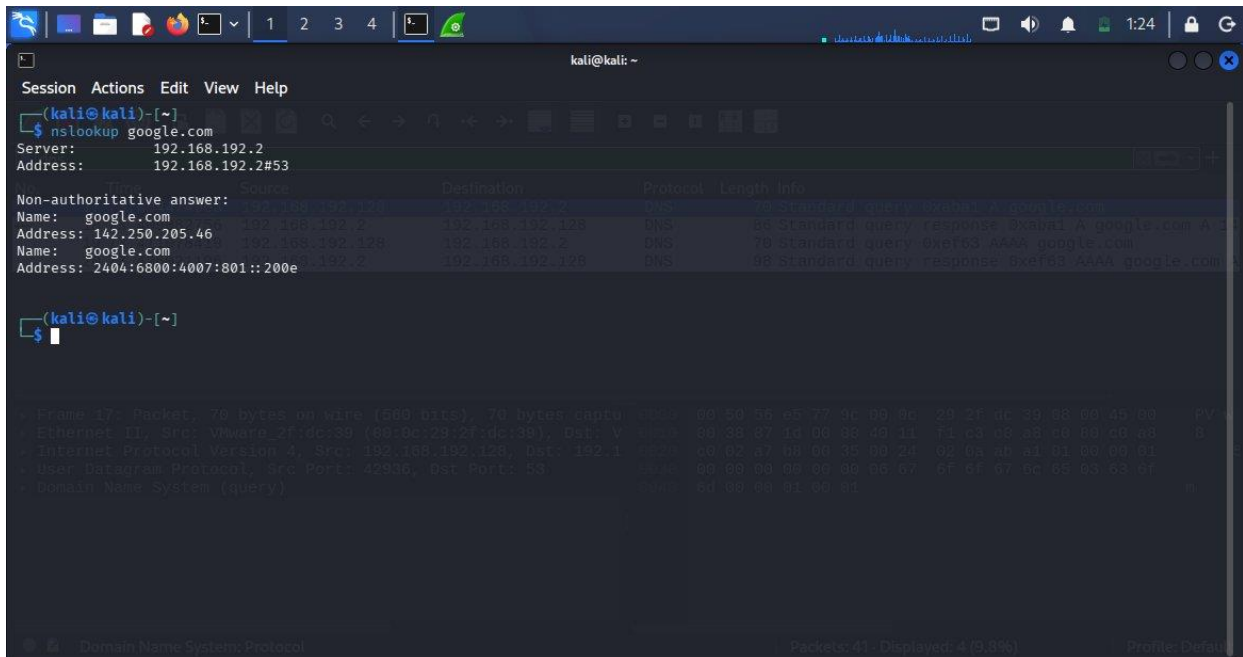


Figure 3: Terminal showing nslookup google.com command with DNS resolution result and Wireshark capturing DNS packets in background

6. Step 4 – DNS Packet Inspection in Wireshark

The display filter "dns" was applied in Wireshark to show only DNS protocol packets. This filtered out all other traffic and displayed only 4 DNS packets out of 31 total captured.

DNS Packets Observed:

- Packet 17: Standard query 0xaba1 A google.com — IPv4 address request
- Packet 18: Standard query response 0xaba1 A google.com A 142.250.205.46 — Response with IP
- Packet 19: Standard query 0xef63 AAAA google.com — IPv6 address request
- Packet 20: Standard query response 0xef63 AAAA google.com — IPv6 response

Packet Layer Analysis (Packet 17):

- Frame: 70 bytes on wire (560 bits)
- Ethernet II: VMware_2f:dc:39 → VMware_a7:b8:00
- Internet Protocol Version 4: Src 192.168.192.128 → Dst 192.168.192.2
- User Datagram Protocol: Src Port 42936 → Dst Port 53 (DNS port)
- Domain Name System: Standard query for google.com

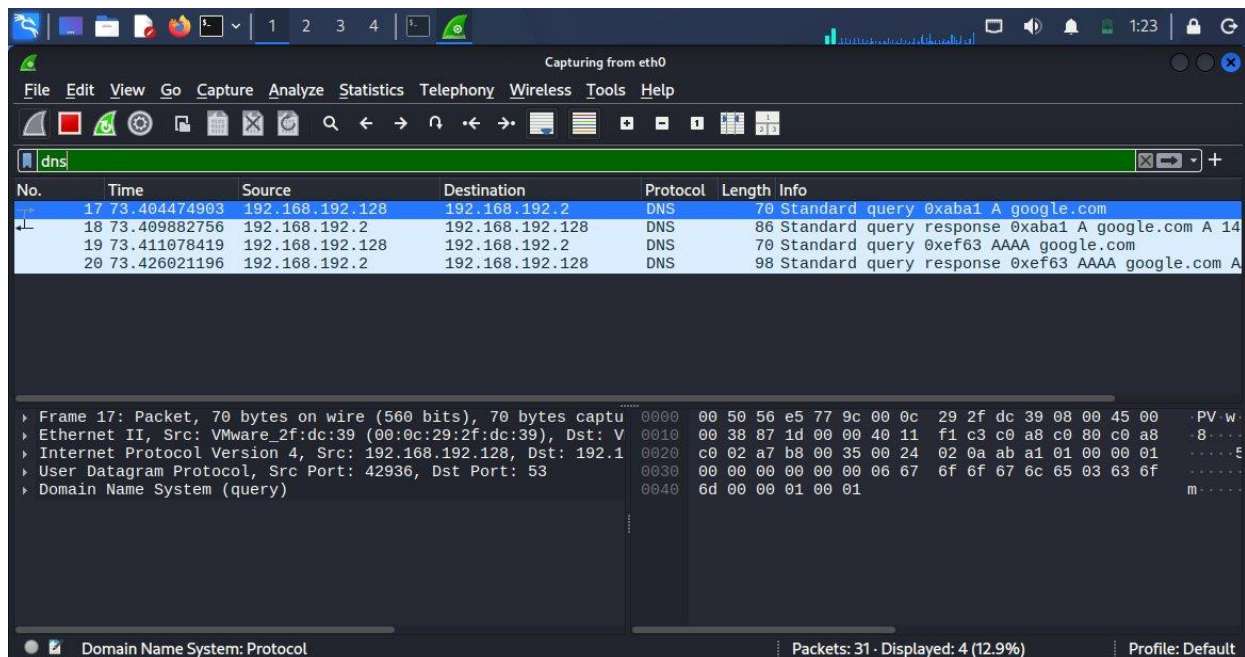


Figure 4: Wireshark DNS filter applied showing 4 DNS packets with packet details panel expanded

7. Step 5 – ICMP Ping Test (Terminal)

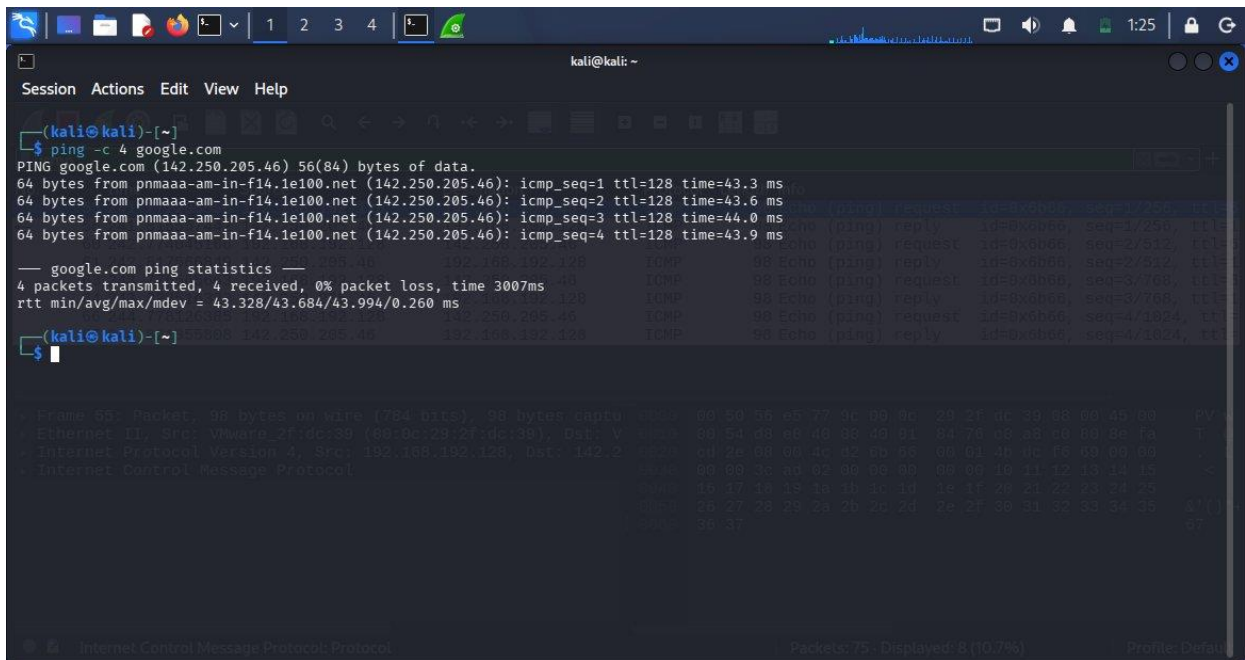
The ping command was executed in the terminal with the `-c 4` flag to send exactly 4 ICMP echo request packets to google.com. This is a standard network connectivity test.

Command executed:

```
$ ping -c 4 google.com
```

Ping Statistics Results:

- Target: google.com (142.250.205.46)
- Packet size: 56(84) bytes of data
- 4 packets transmitted, 4 packets received
- 0% packet loss — excellent network connectivity
- Round-trip time: min=43.3ms / avg=43.684ms / max=43.994ms
- Server response: pnmaaa-am-in-f14.1e100.net (Google server)



```
(kali@kali)-[~]
$ ping -c 4 google.com
PING google.com (142.250.205.46) 56(84) bytes of data:
64 bytes from pnmaaa-am-in-f14.1e100.net (142.250.205.46): icmp_seq=1 ttl=128 time=43.3 ms
64 bytes from pnmaaa-am-in-f14.1e100.net (142.250.205.46): icmp_seq=2 ttl=128 time=43.6 ms
64 bytes from pnmaaa-am-in-f14.1e100.net (142.250.205.46): icmp_seq=3 ttl=128 time=44.0 ms
64 bytes from pnmaaa-am-in-f14.1e100.net (142.250.205.46): icmp_seq=4 ttl=128 time=43.9 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 43.328/43.684/43.994/0.260 ms

(kali@kali)-[~]
$
```

The screenshot shows a Kali Linux terminal window with the command `ping -c 4 google.com` executed. The output displays four successful ICMP echo replies from the Google server (142.250.205.46) with round-trip times ranging from 43.3ms to 44.0ms. Below the replies, the ping statistics are shown, indicating 4 packets transmitted, 4 received, 0% packet loss, and a total time of 3007ms. The round-trip time statistics are: min=43.328ms, avg=43.684ms, max=43.994ms, and mdev=0.260ms. At the bottom of the terminal, a packet capture snippet is visible, showing the details of the first packet: Frame 53, Packet, 98 bytes on wire (784 bits), 98 bytes captured on interface vnet0, Ethernet II, Src: VMware_2f:dc:38 (88:06:23:2f:dc:38), Dst: Vnet0 (08:00:00:00:00:00), Internet Protocol Version 4, Src: 192.168.192.120, Dst: 142.250.205.46, Internet Control Message Protocol.

Figure 5: Terminal showing `ping -c 4 google.com` with successful ICMP echo replies and ping statistics

8. Step 6 – ICMP Packets Captured in Wireshark

With the display filter "icmp" applied in Wireshark, the 8 ICMP packets (4 requests + 4 replies) generated by the ping command were clearly visible. These packets confirm successful two-way communication between the local machine and Google's server.

ICMP Packet Details:

- Protocol: ICMP (Internet Control Message Protocol)
- Source (Requests): 192.168.192.128 (local machine)
- Destination (Requests): 142.250.205.46 (Google server)
- Packet length: 98 bytes each
- Echo (ping) request packets: Packets 55, 60, 63, 66
- Echo (ping) reply packets: Packets 56, 61, 64, 67
- Total packets in session: 110 | ICMP displayed: 8 (7.3%)

Packet Layer Breakdown:

- Frame 55: 98 bytes captured on eth0
- Ethernet II: VMware_2f:dc:39 source MAC
- Internet Protocol Version 4: IPv4 routing information
- Internet Control Message Protocol: ICMP echo request data

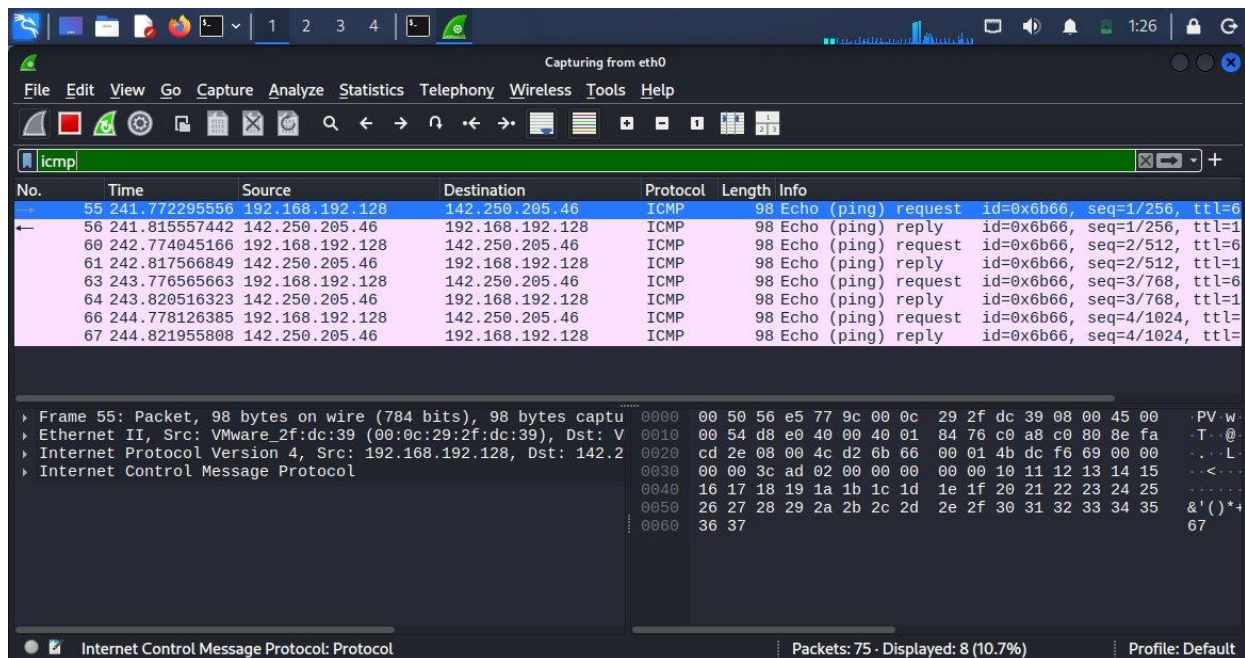


Figure 6: Wireshark ICMP filter showing 8 Echo request/reply packets captured during ping test

9. Step 7 – Protocol Hierarchy Statistics

The Protocol Hierarchy Statistics window (accessed from Statistics menu) provides a tree-view breakdown of all captured protocols and their relative usage in terms of packet count and bytes.

Protocol Hierarchy Results (ICMP filter active):

Protocol	Full Name	Purpose	Packets Captured
ARP	Address Resolution Protocol	Maps IP address to MAC address	19+
DNS	Domain Name System	Resolves domain names to IP addresses	4
ICMP	Internet Control Message Protocol	Sends ping/echo requests & replies	8
UDP	User Datagram Protocol	Transport layer for DNS queries	4
IPv4	Internet Protocol Version 4	Logical addressing & routing	8+

Key Observations:

- Frame → Ethernet → IPv4 → ICMP is the complete protocol stack
- Total: 8 packets, 784 bytes captured
- ICMP accounted for 100% of displayed packets
- ICMP used 65.3% of total bytes (512 bytes / 784 total)
- Ethernet headers: 14.3% (112 bytes)
- IPv4 headers: 20.4% (160 bytes)
- Transmission rate: 2,055 bits/second

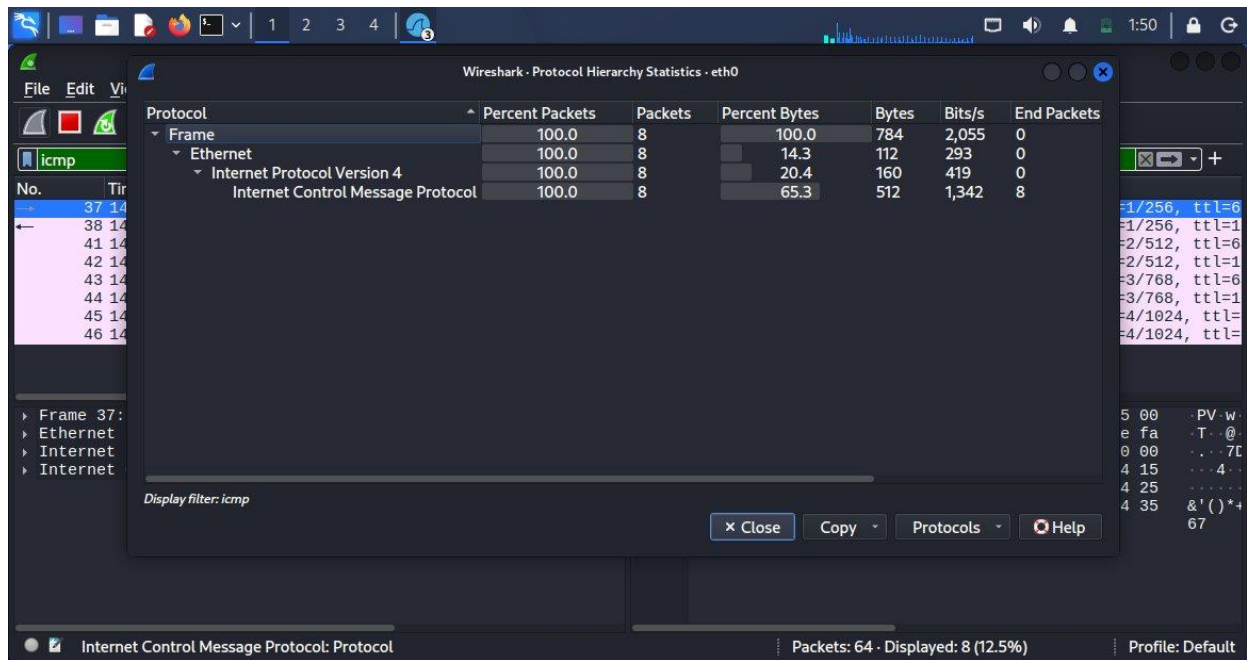


Figure 7: Wireshark Protocol Hierarchy Statistics showing complete protocol breakdown for ICMP capture

10. Capture Stopped & Session Summary

After completing all analysis steps, the capture was stopped using the red stop button in Wireshark toolbar. The final session showed a complete record of all captured packets.

Final Session Statistics:

- Total packets captured: 110
- ICMP packets displayed: 8 (7.3% of total)
- Dropped packets: 0 (0.0%) — no data loss
- Capture file: wireshark_eth0B8LNO3.pcapng
- Interface: eth0 on Kali Linux

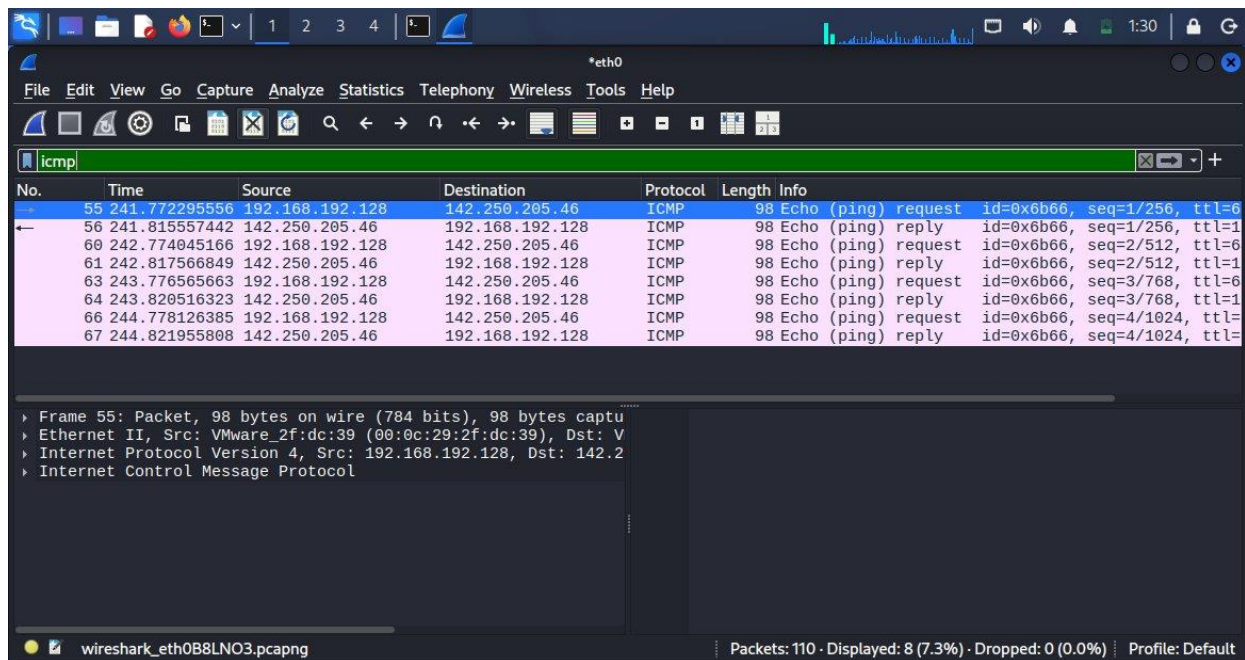


Figure 8: Wireshark capture stopped showing final session statistics with 110 total packets

11. Observations & Results

The following key observations were made during this practical:

DNS Analysis

- DNS operates on UDP port 53
- nslookup generated both A (IPv4) and AAAA (IPv6) record queries
- DNS resolution was completed in milliseconds
- The local DNS server (192.168.192.2) forwarded queries to Google's DNS

ICMP / Ping Analysis

- All 4 ping packets were successfully sent and received (0% packet loss)
- Average round-trip time of 43.68ms — normal for internet connection
- ICMP uses no port numbers — it is a Layer 3 protocol
- Each request was matched with exactly one reply from Google's server

Protocol Observations

- ARP is the most frequent protocol in a local network — runs constantly
- Network stack follows OSI model: Physical → Data Link → Network → Transport → Application

12. Conclusion

This practical successfully demonstrated the use of Wireshark for network traffic analysis on Kali Linux. Through this experiment, the following was learned and achieved:

- Wireshark can capture and display real-time network packets from any interface
- Display filters (dns, icmp, tcp, arp) allow focused analysis of specific protocols
- DNS protocol uses UDP and resolves human-readable domain names to IP addresses
- ICMP protocol is used for network diagnostics — ping command generates ICMP packets
- Protocol Hierarchy gives a clear view of network traffic distribution
- Kali Linux provides an excellent environment for network security analysis

This practical provided hands-on experience with packet-level network analysis which is fundamental to understanding cybersecurity and networking concepts.
